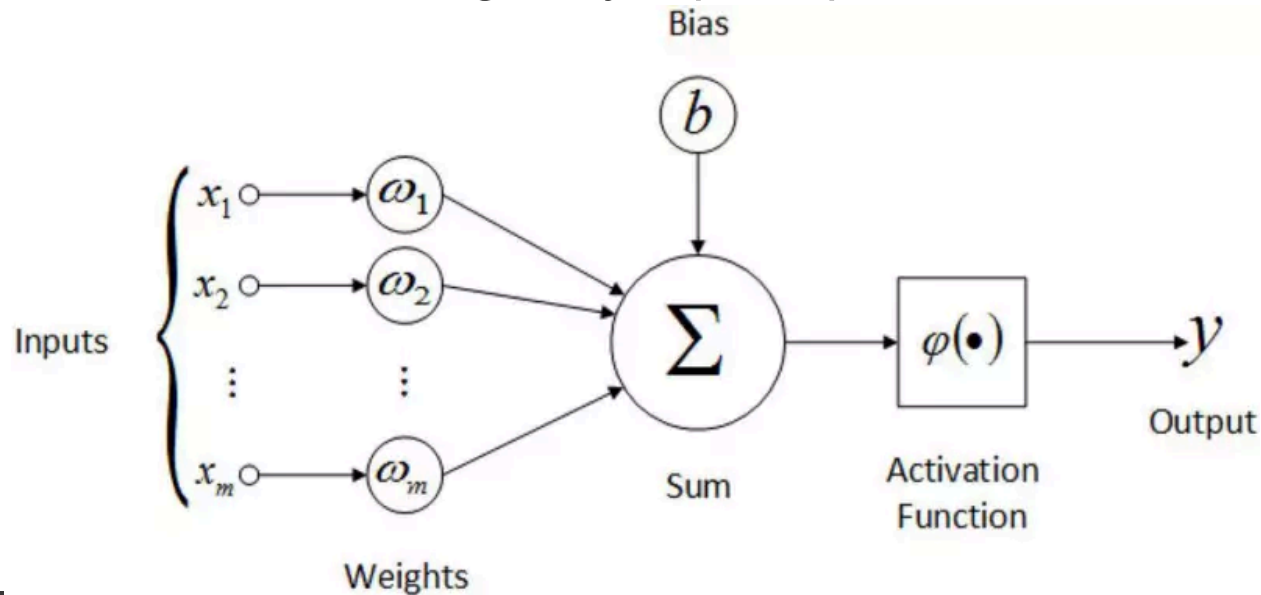


Feed-Forward Neural Network and how does it work?

A Feed-Forward Neural Network is a single layer perceptron. A sequence of inputs enter the layer and are multiplied by the weights in this model. The weighted input values are then summed together to form a total. If the sum of the values is more than a predetermined threshold, which is normally set at zero, the output value is usually 1, and if the sum is less than the threshold, the output value is usually -1. The single-layer perceptron is a popular feed-forward neural network model that is frequently used for classification. Single-layer perceptrons can also contain machine



learning features.

For example in a credit scoring system, banks use an FNN which analyze users financial profiles such as income, credit history and spending habits to determine their creditworthiness.

Each piece of information flows through the network's layers where various calculations are made to produce a final score.

Structure of a Feedforward Neural Network

Feedforward Neural Networks have a structured layered design where data flows sequentially through each layer.

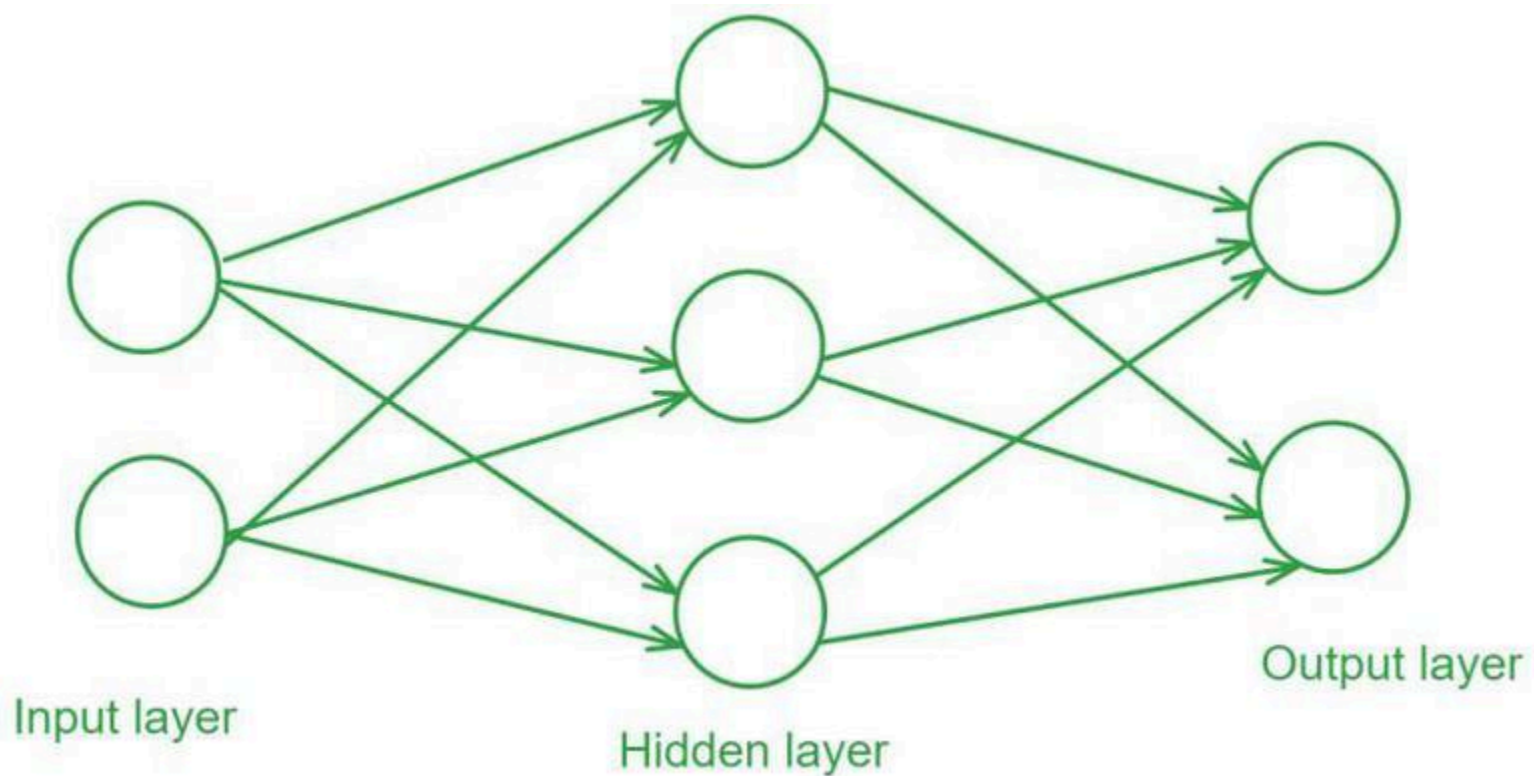
1. Input Layer: The input layer consists of neurons that receive the input data.

Each neuron in the input layer represents a feature of the input data.

2. Hidden Layers: One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.

3. Output Layer: The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



Feed Forward Neural Network

Activation Functions

Activation functions introduce non-linearity into the network enabling it to learn and model complex data patterns.

Common activation functions include:

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- ReLU: $\text{ReLU}(x) = \max(0, x)$

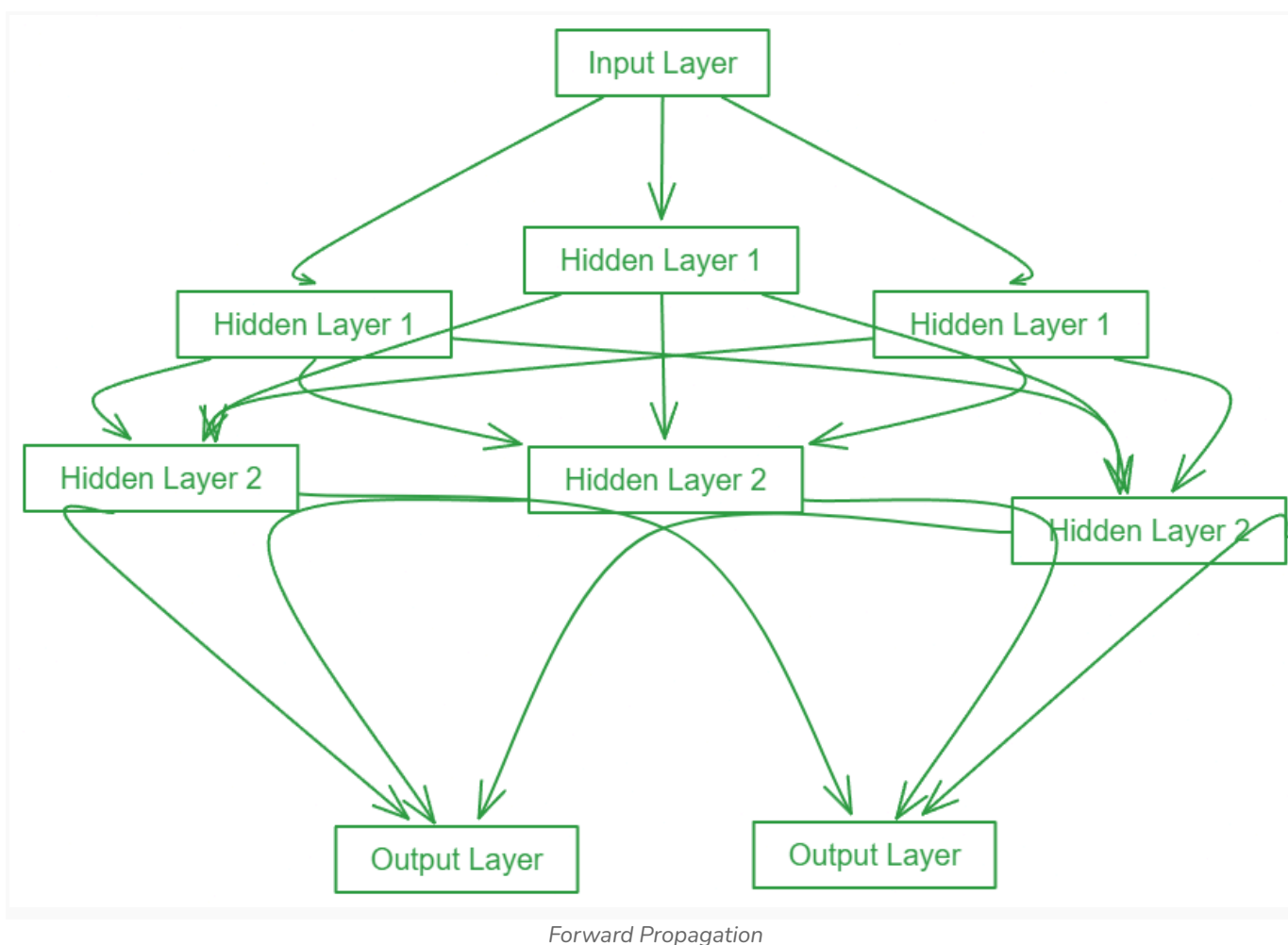
Training a Feedforward Neural Network

Training a Feedforward Neural Network involves adjusting the weights of the neurons to minimize the error between the predicted output and the actual output. This process is typically performed using backpropagation and gradient descent.

1. **Forward Propagation:** During forward propagation the input data passes through the network and the output is calculated.

2. **Loss Calculation:** The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
3. **Backpropagation:** In backpropagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.

Note: *During training, a feedforward neural network performs forward pass followed by backpropagation to update weights, while during prediction only the forward pass is used.*



Gradient Descent

Gradient Descent is an optimization algorithm used to minimize the loss function by iteratively updating the weights in the direction of the negative gradient. Common variants of gradient descent include:

- **Batch Gradient Descent:** Updates weights after computing the gradient over the entire dataset.

- **Stochastic Gradient Descent (SGD):** Updates weights for each training example individually.
- **Mini-batch Gradient Descent:** It Updates weights after computing the gradient over a small batch of training examples.

Evaluation of Feedforward neural network

Evaluating the performance of the trained model involves several metrics:

- **Accuracy:** The proportion of correctly classified instances out of the total instances.
- **Precision:** The ratio of true positive predictions to the total predicted positives.
- **Recall:** The ratio of true positive predictions to the actual positives.
- **F1 Score:** The harmonic mean of precision and recall, providing a balance between the two.

- **Confusion Matrix:** A table used to describe the performance of a classification model, showing the true positives, true negatives, false positives and false negatives.

Implementation of Feedforward Neural Network

This code demonstrates the process of building, training and evaluating a neural network model using **TensorFlow and Keras** to classify handwritten digits from the MNIST dataset.

The model architecture is defined using the Sequential consisting of:

- a Flatten layer to convert the 2D image input into a 1D array
- a Dense layer with 128 neurons and ReLU activation
- a final Dense layer with 10 neurons and softmax activation to output probabilities for each digit class.

Model is compiled with

- Adam optimizer
- Sparse Categorical Crossentropy loss function
- Sparse Categorical Accuracy metric
- Then trained for 5 epochs on the training data

Applications of the Feed Forward Neural Networks

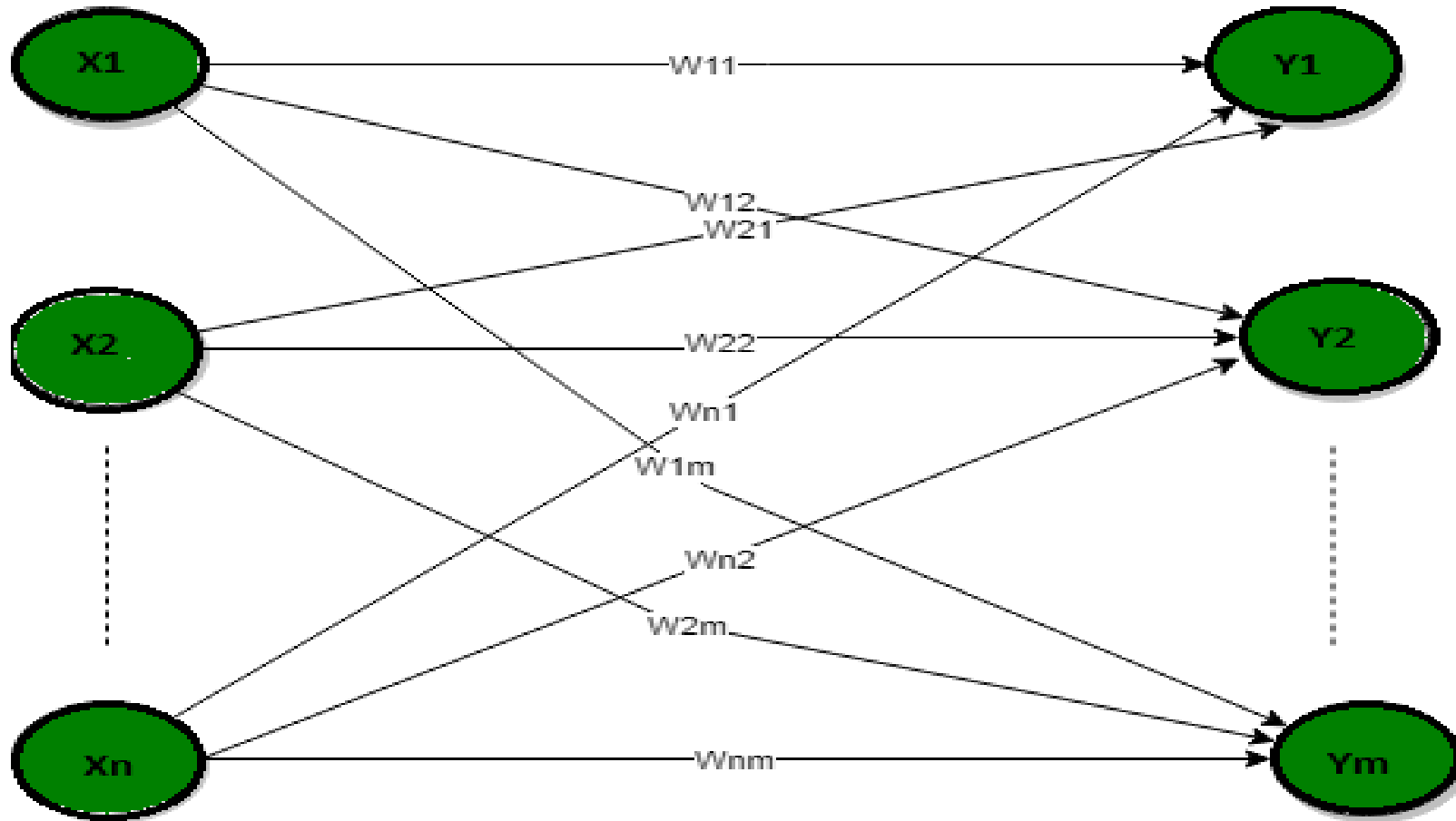
A Feed Forward Neural Network is an artificial neural network in which the nodes are connected circularly.

A feed-forward neural network, in which some routes are cycled, is the polar opposite of a recurrent neural network.

The feed-forward model is the simplest type of neural network because the input is only processed in one direction.

The data always flows in one direction and never backwards, regardless of how many buried nodes it passes through.

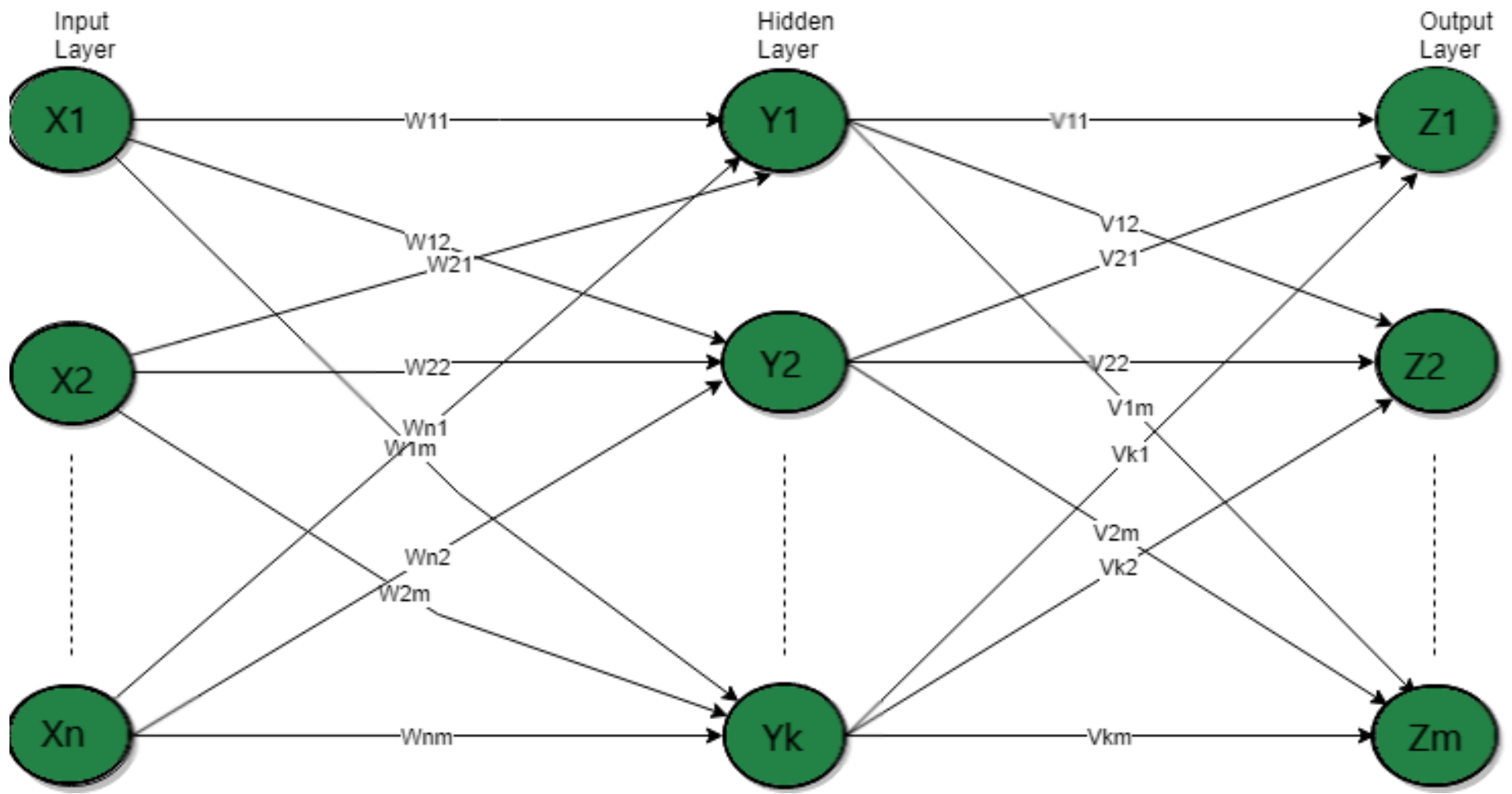
1. Single-layer feed-forward network



In this type of network, we have only two layers, the input layer and the output layer but the input layer does not count because no computation is performed in this layer.

The output layer is formed when different weights are applied to input nodes and the cumulative effect per node is taken. After this, the neurons collectively give the output layer to compute the output signals.

2. Multilayer feed-forward network



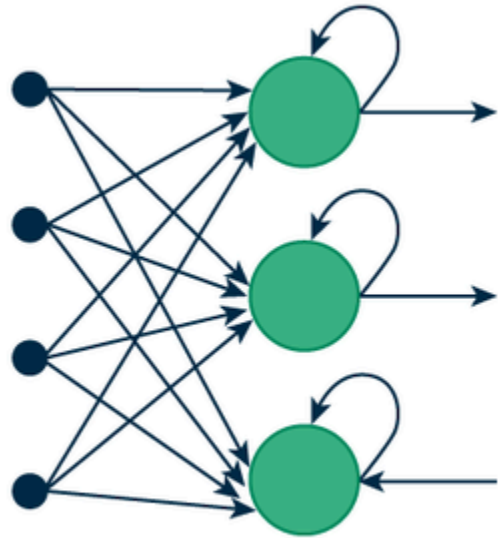
This layer also has a hidden layer that is internal to the network and has no direct contact with the external layer. The existence of one or more hidden layers enables the

network to be computationally stronger, a feed-forward network because of information flow through the input function, and the intermediate computations used to determine the output Z . There are no feedback connections in which outputs of the model are fed back into itself.

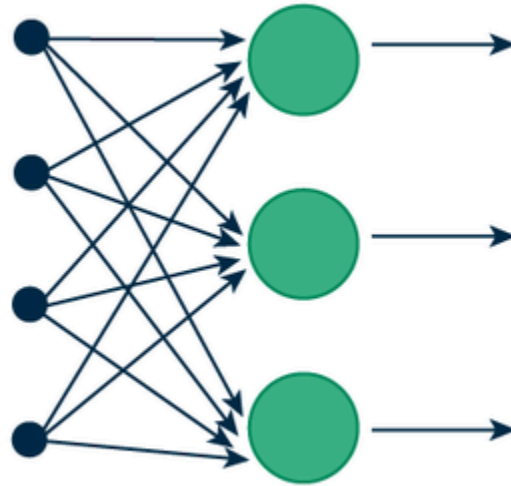
3. Recurrent neural network (RNN)

A recurrent neural network (RNN) is a type of neural network designed to process sequential data by maintaining a "memory" of past inputs. Unlike traditional feedforward networks, RNNs incorporate feedback loops, allowing information to be passed from one processing step to the next. This enables them to capture dependencies and patterns in sequences, making them

suitable for tasks like natural language processing, time series analysis, and speech recognition.



(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

Sequential Data:

RNNs are specifically designed for data where the order of elements matters. Examples include text, time series data, and audio signals.

Feedback Loops:

The core idea of an RNN is the presence of feedback loops. This means that the output of a neuron (or a layer of

neurons) at one time step is fed back as input to the same neuron (or layer) at the next time step.

Memory:

This feedback mechanism effectively gives the network a "memory" of past inputs, allowing it to consider the context of previous steps when processing the current input.

Applications:

RNNs are used in a wide range of applications, including:

Natural Language Processing (NLP): Text generation, machine translation, sentiment analysis, and speech recognition.

Time Series Analysis: Forecasting stock prices, weather patterns, and other time-dependent data.

Image Captioning: Generating descriptions for images based on their visual content.

Challenges:

While powerful, RNNs can suffer from issues like vanishing gradients (where the network struggles to learn long-term dependencies) and exploding gradients (where the network becomes unstable during training).

Variations:

To address these challenges, various modifications to the basic RNN architecture have been developed, such as:

Long Short-Term Memory (LSTM): A type of RNN with a more sophisticated memory mechanism that helps mitigate vanishing gradients.

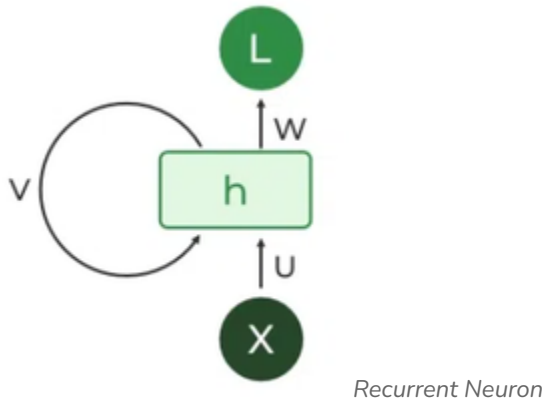
Gated Recurrent Unit (GRU): A simplified version of LSTM, often with better performance in certain tasks.

Key Components of RNNs

There are mainly two components of RNNs that we will discuss.

1. Recurrent Neurons

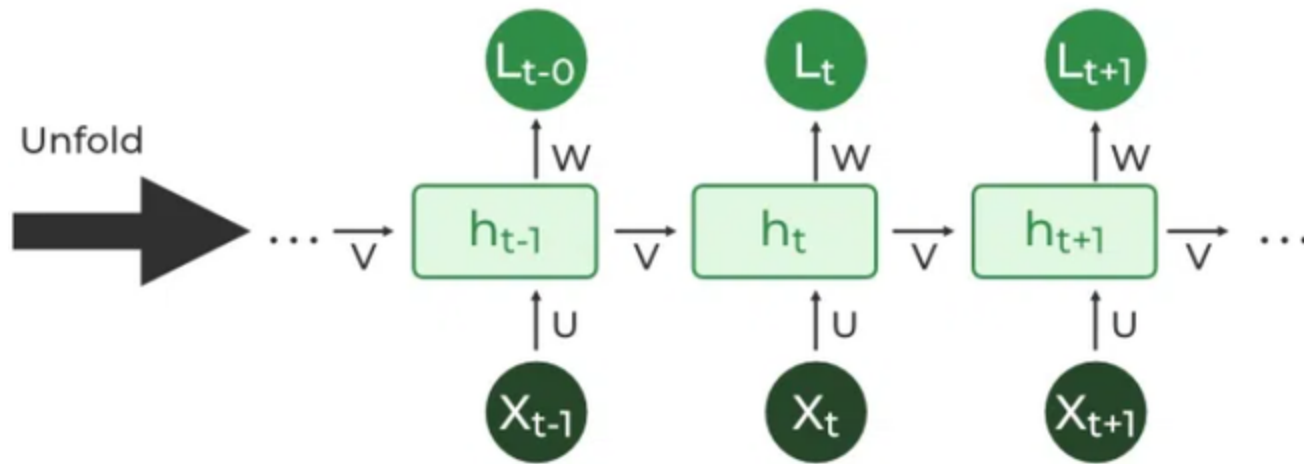
The fundamental processing unit in RNN is a **Recurrent Unit**. They hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



2. RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables [backpropagation through time \(BPTT\)](#) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



RNN Unfolding

Recurrent Neural Network Architecture

RNNs share similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output. Unlike traditional deep neural networks where each dense layer has distinct weight matrices. RNNs use shared weights across time steps, allowing them to remember information over sequences.

In RNNs the hidden state

h_i

H

i

is calculated for every input

x_i

X

i

to retain sequential dependencies. The computations follow these core formulas:

1. Hidden State Calculation:

$$h=\sigma(U \cdot X+W \cdot h_{t-1}+B)$$

$$h=\sigma(U \cdot X+W \cdot h$$

$$_{t-1}$$

$$+B)$$

Here:

- h
- h represents the current hidden state.
- U
- U and
- W
- W are weight matrices.
- B
- B is the bias.

2. Output Calculation:

$$Y=O(V \cdot h+C)$$

$$Y=O(V \cdot h+C)$$

The output

Y

Y is calculated by applying

O

O an activation function to the weighted hidden state where

V

V and

C

C represent weights and bias.

3. Overall Function:

$$Y=f(X,h,W,U,V,B,C)$$

$$Y=f(X,h,W,U,V,B,C)$$

This function defines the entire RNN operation where the state matrix

S

S holds each element

s_i

s

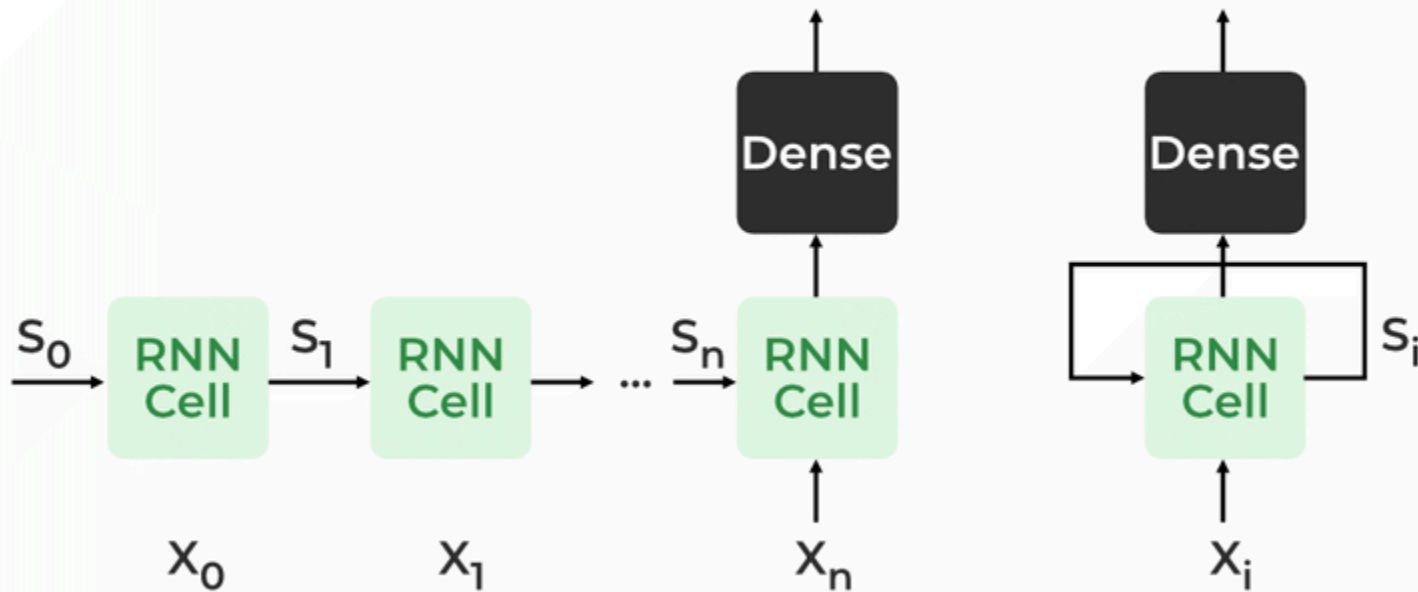
i

representing the network's state at each time step

i

i .

RECURRENT NEURAL NETWORKS



How does RNN work?

At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps. This memory allows the network to store past knowledge and adapt based on new inputs.

Updating the Hidden State in RNNs

The current hidden state h_t depends on the previous state h_{t-1} and the current input x_t and is calculated using the following relations:

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

2. Activation Function Application:

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

3. Output Calculation:

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

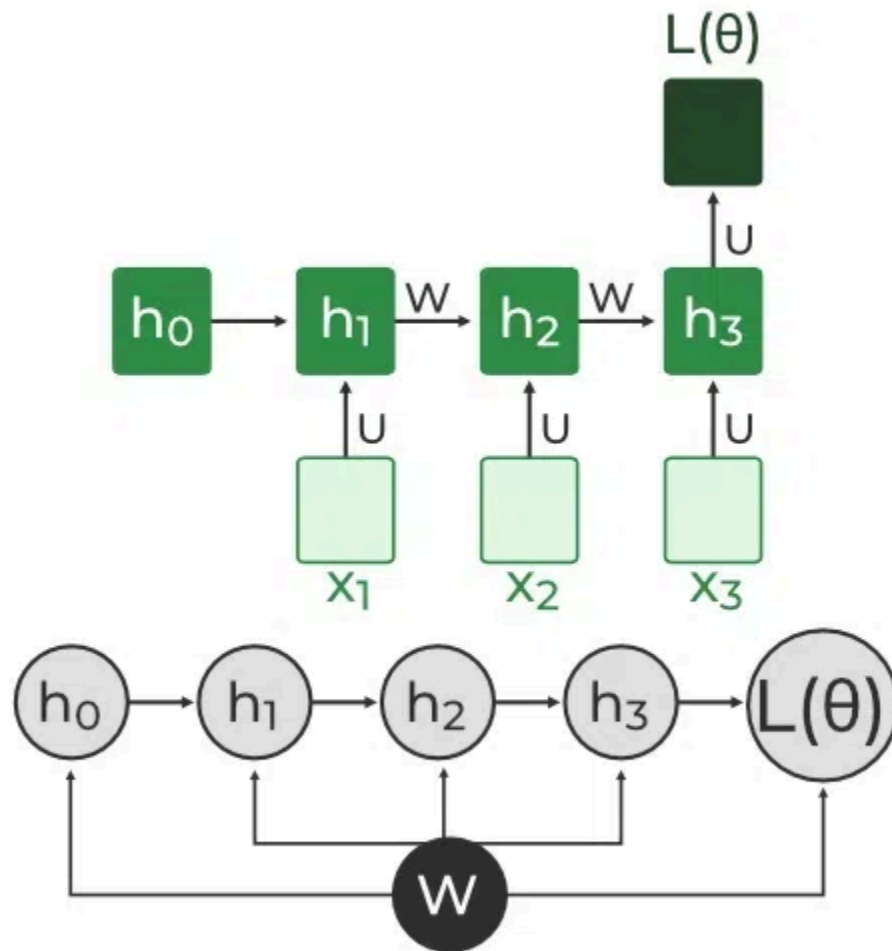
These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as **backpropagation through time**.

Backpropagation Through Time (BPTT) in RNNs

Backpropagation Through Time (BPTT) in RNNs

Since RNNs process sequential data Backpropagation Through Time (BPTT) is used to update the network's parameters. The loss function $L(\theta)$ depends on the final hidden state h_3 and each hidden state relies on preceding ones forming a sequential dependency chain:

h_3 depends on h_2 , h_2 depends on h_1 , $\dots$, h_1 depends on h_0 .



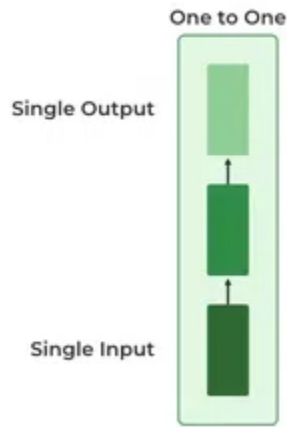
In BPTT, gradients are backpropagated through each time step. This is essential for updating network parameters based on temporal dependencies.

Types Of Recurrent Neural Networks

There are four types of RNNs based on the number of inputs and outputs in the network:

1. One-to-One RNN

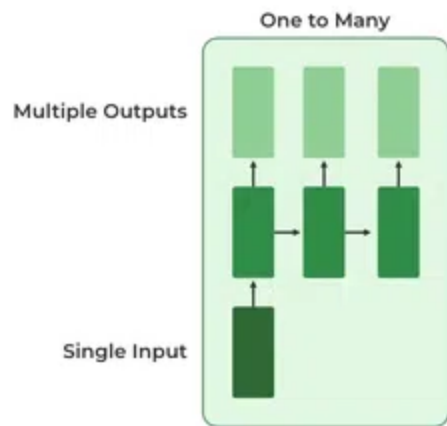
This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



One to One RNN

2. One-to-Many RNN

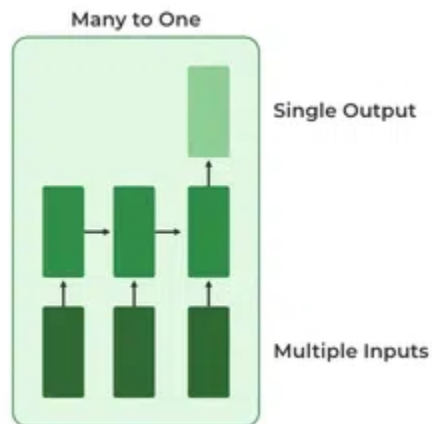
In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words as a caption.



One to Many RNN

3. Many-to-One RNN

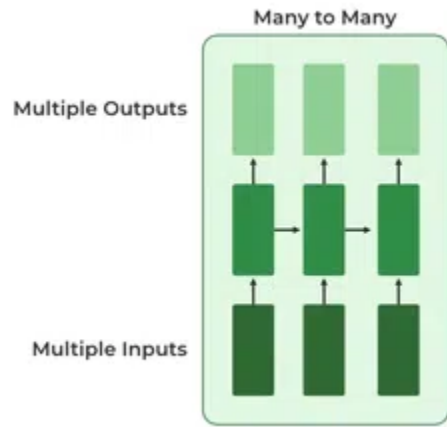
The **Many-to-One RNN** receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



Many to One RNN

4. Many-to-Many RNN

The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Many to Many RNN

Variants of Recurrent Neural Networks (RNNs)

There are several variations of RNNs, each designed to address specific challenges or optimize for certain tasks:

1. Vanilla RNN

This simplest form of RNN consists of a single hidden layer where weights are shared across time steps. Vanilla RNNs are suitable for learning short-term dependencies but are limited by the vanishing gradient problem, which hampers long-sequence learning.

2. Bidirectional RNNs

Bidirectional RNNs process inputs in both forward and backward directions, capturing both past and future context for each time step. This architecture is ideal for tasks where the entire sequence is available, such as named entity recognition and question answering.

3. Long Short-Term Memory Networks (LSTMs)

Long Short-Term Memory Networks (LSTMs) introduce a memory mechanism to overcome the vanishing gradient problem. Each LSTM cell has three gates:

- **Input Gate:** Controls how much new information should be added to the cell state.
- **Forget Gate:** Decides what past information should be discarded.
- **Output Gate:** Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical.

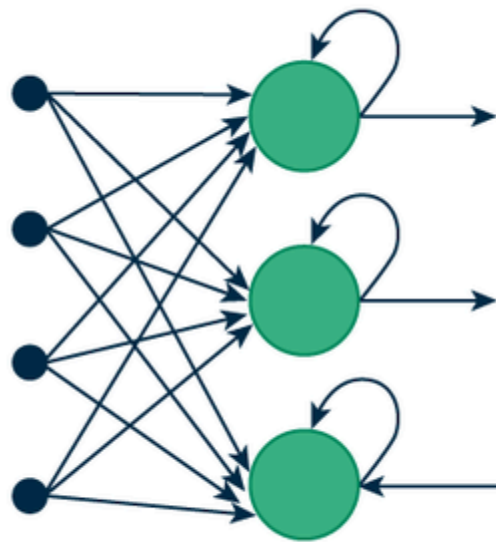
4. Gated Recurrent Units (GRUs)

Gated Recurrent Units (GRUs) simplify LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism. This design is computationally efficient, often performing similarly to LSTMs and is useful in tasks where simplicity and faster training are beneficial.

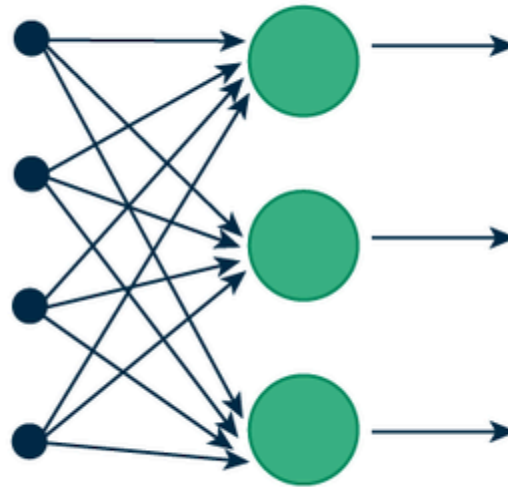
How RNN Differs from Feedforward Neural Networks?

Feedforward Neural Networks (FNNs) process data in one direction from input to output without retaining information from previous inputs. This makes them suitable for tasks with independent inputs like image classification. However FNNs struggle with sequential data since they lack memory.

Recurrent Neural Networks (RNNs) solve this **by incorporating loops that allow information from previous steps to be fed back into the network**. This feedback enables RNNs to remember prior inputs making them ideal for tasks where context is important.



(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

Recurrent Vs Feedforward networks

Implementing a Text Generator Using Recurrent Neural Networks (RNNs)

In this section, we create a character-based text generator using Recurrent Neural Network (RNN) in TensorFlow and Keras. We'll implement an RNN that learns patterns from a text sequence to generate new text character-by-character.

1. Importing Necessary Libraries

We start by importing essential libraries for data handling and building the neural network.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
```

2. Defining the Input Text and Prepare Character Set

We define the input text and identify unique characters in the text which we'll encode for our model.

```
text = "This is GeeksforGeeks a software training institute"
chars = sorted(list(set(text)))
```

```
char_to_index = {char: i for i, char in enumerate(chars)}  
index_to_char = {i: char for i, char in enumerate(chars)}
```

3. Creating Sequences and Labels

To train the RNN, we need sequences of fixed length (seq_length) and the character following each sequence as the label.

```
seq_length = 3  
sequences = []  
labels = []  
  
for i in range(len(text) - seq_length):  
    seq = text[i:i + seq_length]  
    label = text[i + seq_length]  
    sequences.append([char_to_index[char] for char in seq])  
    labels.append(char_to_index[label])  
  
X = np.array(sequences)  
y = np.array(labels)
```

4. Converting Sequences and Labels to One-Hot Encoding

For training we convert X and y into one-hot encoded tensors.

```
X_one_hot = tf.one_hot(X, len(chars))  
y_one_hot = tf.one_hot(y, len(chars))
```

5. Building the RNN Model

We create a simple RNN model with a hidden layer of 50 units and a Dense output layer with [softmax activation](#).

```
model = Sequential()  
model.add(SimpleRNN(50, input_shape=(seq_length, len(chars)), activation='relu'))
```



```
model.add(Dense(len(chars), activation='softmax'))
```

6. Compiling and Training the Model

We compile the model using the categorical_crossentropy loss and train it for 100 epochs.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_one_hot, y_one_hot, epochs=100)
```

Output:

```
2/2 ————— 0s 29ms/step - accuracy: 0.9618 - loss: 0.0664
Epoch 94/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0729
Epoch 95/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0661
Epoch 96/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0655
Epoch 97/100
2/2 ————— 0s 28ms/step - accuracy: 0.9514 - loss: 0.0652
Epoch 98/100
2/2 ————— 0s 26ms/step - accuracy: 0.9618 - loss: 0.0648
Epoch 99/100
2/2 ————— 0s 29ms/step - accuracy: 0.9618 - loss: 0.0590
Epoch 100/100
2/2 ————— 0s 26ms/step - accuracy: 0.9618 - loss: 0.0583
<keras.src.callbacks.history.History at 0x7a5e40a739d0>
```

Training the RNN model

7. Generating New Text Using the Trained Model

After training we use a starting sequence to generate new text character by character.

```
start_seq = "This is G"
generated_text = start_seq
```

```
for i in range(50):
    x = np.array([[char_to_index[char] for char in generated_text[-seq_length:]]])
    x_one_hot = tf.one_hot(x, len(chars))
    prediction = model.predict(x_one_hot)
```

```
next_index = np.argmax(prediction)
next_char = index_to_char[next_index]
generated_text += next_char
```

```
print("Generated Text:")
print(generated_text)
```

Output:

```
1/1 ————— 0s 37ms/step
1/1 ————— 0s 36ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 39ms/step
Generated Text:
This is Geeks a software training institutetraining institu
```

Predicting the next word

Advantages of Recurrent Neural Networks

- **Sequential Memory:** RNNs retain information from previous inputs making them ideal for time-series predictions where past data is crucial.
- **Enhanced Pixel Neighborhoods:** RNNs can be combined with convolutional layers to capture extended pixel neighborhoods improving performance in image and video data processing.

Limitations of Recurrent Neural Networks (RNNs)

While RNNs excel at handling sequential data they face two main training challenges i.e [vanishing gradient and exploding gradient problem](#):

1. **Vanishing Gradient:** During backpropagation gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn long-term dependencies which is crucial for tasks like language translation.
2. **Exploding Gradient:** Sometimes gradients grow uncontrollably causing excessively large weight updates that de-stabilize training.

These challenges can hinder the performance of standard RNNs on complex, long-sequence tasks.

Applications of Recurrent Neural Networks

RNNs are used in various applications where data is sequential or time-based:

- **Time-Series Prediction:** RNNs excel in forecasting tasks, such as stock market predictions and weather forecasting.
- **Natural Language Processing (NLP):** RNNs are fundamental in NLP tasks like language modeling, sentiment analysis and machine translation.
- **Speech Recognition:** RNNs capture temporal patterns in speech data, aiding in speech-to-text and other audio-related applications.
- **Image and Video Processing:** When combined with convolutional layers, RNNs help analyze video sequences, facial expressions and gesture recognition.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.metrics import SparseCategoricalAccuracy

# Load and prepare the MNIST dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# Build the model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer=Adam(),
              loss=SparseCategoricalCrossentropy(),
              metrics=[SparseCategoricalAccuracy()])

# Train the model
model.fit(x_train, y_train, epochs=5)

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f'\nTest accuracy: {test_acc}')
```

Output:

Test accuracy: 0.9767000079154968

